100 XP

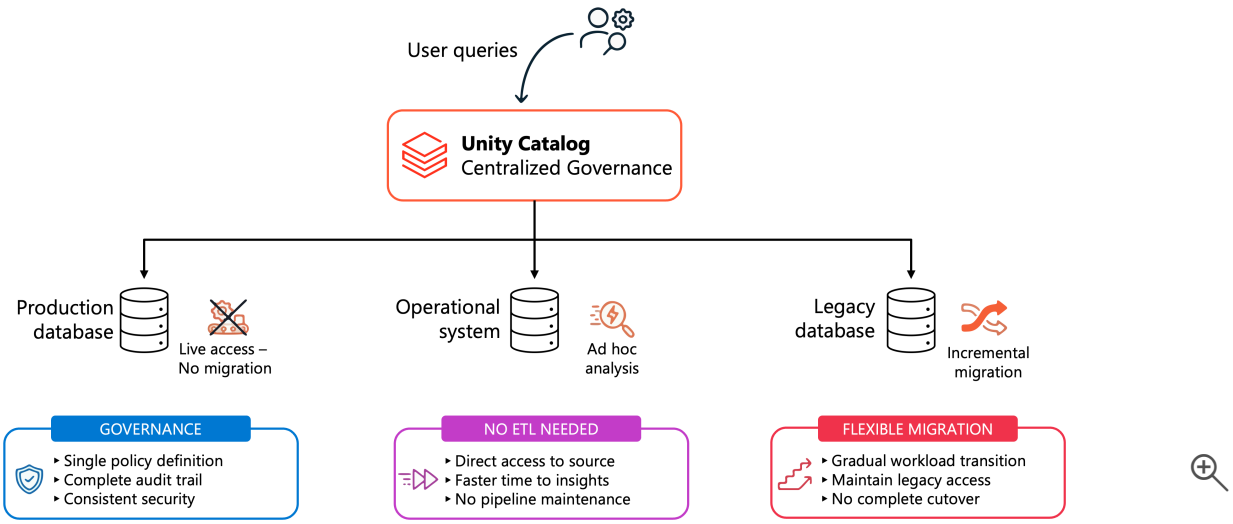
Implement foreign catalog

11 minutes

Your analytics team needs immediate access to customer transaction data stored in a PostgreSQL database. Moving terabytes of operational data into Azure Databricks for exploratory analysis would take weeks and consume significant resources. At the same time, your organization requires **centralized governance** for all data access, regardless of where the data resides. **Foreign catalogs in Unity Catalog** solve this challenge by enabling you to query external databases directly while maintaining Unity Catalog's access control, lineage tracking, and audit capabilities.

Why implement foreign catalogs?

Foreign catalogs extend Unity Catalog's governance to external data systems without requiring data migration. This approach becomes essential when you need to balance operational efficiency with security and compliance requirements.



Governance and auditability remain consistent across your data estate. With foreign catalogs, your security team defines access policies once in Unity Catalog, and those policies apply whether

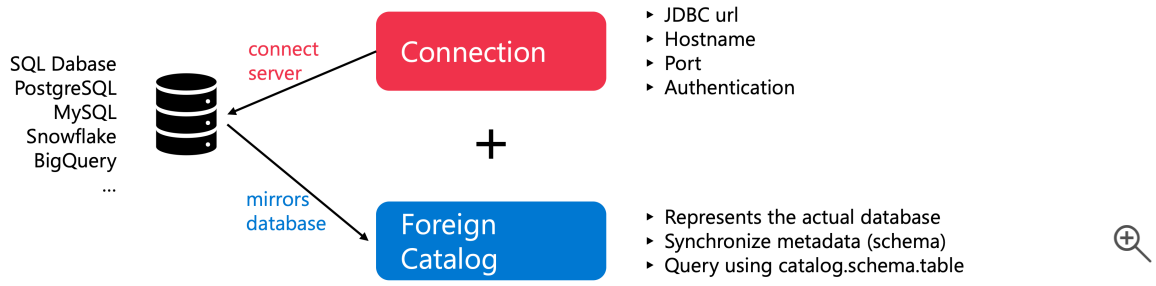
users query data in Databricks-managed tables or external databases. Every query against foreign data appears in Unity Catalog's audit logs, providing complete visibility into who accessed what data and when. This centralized governance eliminates the complexity of managing permissions across multiple systems.

External data access without ETL accelerates time to insights. You can run proof-of-concept analyses on production databases, create ad hoc reports from live operational systems, or validate data quality before committing to a full migration. The data stays in its original location, avoiding the time and cost of building and maintaining ETL pipelines for exploratory work.

Incremental migration support provides flexibility during transitions to Unity Catalog. Organizations moving from legacy systems can implement foreign catalogs to maintain access to data that hasn't migrated yet, allowing workloads to transition gradually rather than requiring a complete cutover.

Understand connections and foreign catalogs

A **connection** and a **foreign catalog** work together to enable external data access, but they serve different purposes in the architecture. Understanding this relationship is essential before you begin implementation.



A **connection** stores the technical details needed to reach an external database system. Think of it as a registered credential that specifies where the database lives and how to authenticate to it. The connection contains the JDBC URL, hostname, port, and authentication credentials for systems like PostgreSQL, MySQL, Snowflake, or BigQuery. You create the connection once and can use it to establish multiple foreign catalogs if needed.

A **foreign catalog** represents the actual database you want to query. It mirrors the structure of an external database within Unity Catalog's namespace, making external schemas and tables accessible through standard SQL queries. When you create a foreign catalog, you specify which

connection to use and which external database to mirror. Unity Catalog then synchronizes the metadata, allowing you to reference external tables using the familiar `catalog.schema.table` naming pattern.

This separation between connection and foreign catalog provides important flexibility. A single connection to your PostgreSQL server might support multiple foreign catalogs, each representing a different database on that server. The connection handles authentication, while the foreign catalog determines what data becomes visible in Unity Catalog.

Configure a connection to the external database

Before you can create a foreign catalog, you must establish a connection that defines how Azure Databricks reaches your external database. The connection stores credentials securely and validates network connectivity to the target system.

Prerequisites must be in place before creating a connection. You need either the **CREATE CONNECTION** privilege on the Unity Catalog metastore or metastore admin permissions. Your Azure Databricks compute must use Runtime 13.3 LTS or above with Standard or Dedicated access mode. Network connectivity between your Azure Databricks workspace and the external database is essential; firewalls and network security groups must allow traffic on the database port.

Using SQL, you create a connection by specifying the database type and configuration options. This example creates a connection to a PostgreSQL database:

SQL

```sql
CREATE CONNECTION postgresql_prod TYPE postgresql
OPTIONS (
  host 'prod-db.example.com',
  port '5432',
  user secret ('prod-secrets','postgres-user'),
  password secret ('prod-secrets','postgres-password')
);
```

This command establishes a connection named `postgresql_prod` that points to a PostgreSQL server. The `secret` function references credentials stored in Databricks secrets rather than embedding passwords directly in the SQL statement, following security best practices. Each database type (MySQL, Snowflake, BigQuery) requires specific options appropriate to that system.

For **Azure SQL Database**, you would create a connection like this:

```sql
CREATE CONNECTION azuresql_prod TYPE SQLSERVER
OPTIONS (
  host 'myserver.database.windows.net',
  port '1433',
  user secret ('prod-secrets','azuresql-user'),
  password secret ('prod-secrets','azuresql-password')
);
```

Using Catalog Explorer, you can create connections through the Azure Databricks interface:

1. In the navigation menu, select **Catalog**.
2. Select the **Add** icon and choose **Add a connection**.
3. Enter a name for your connection, such as `postgresql_prod`.
4. Select the connection type (for example, PostgreSQL, MySQL, or Snowflake).
5. Provide the required connection details including host, port, and credentials.
6. (Optional) Select **Test connection** to verify connectivity.
7. Select **Create connection**.

Validation confirms that Azure Databricks can reach your database. After creating the connection, test it by attempting a simple metadata query. If the connection fails, verify network rules, credentials, and that the database server accepts connections from Azure Databricks IP addresses.

Create a foreign catalog using the connection

With a validated connection in place, you're ready to create the foreign catalog that makes external tables accessible through Unity Catalog. This step establishes the mapping between your external database and Unity Catalog's namespace.

Permissions required include the **CREATE CATALOG** privilege on the metastore and either ownership of the connection or the **CREATE FOREIGN CATALOG** privilege on that specific connection. These granular permissions allow you to control who can expose external data through Unity Catalog.

Using SQL, you create a foreign catalog by referencing the connection and specifying which external database to mirror:

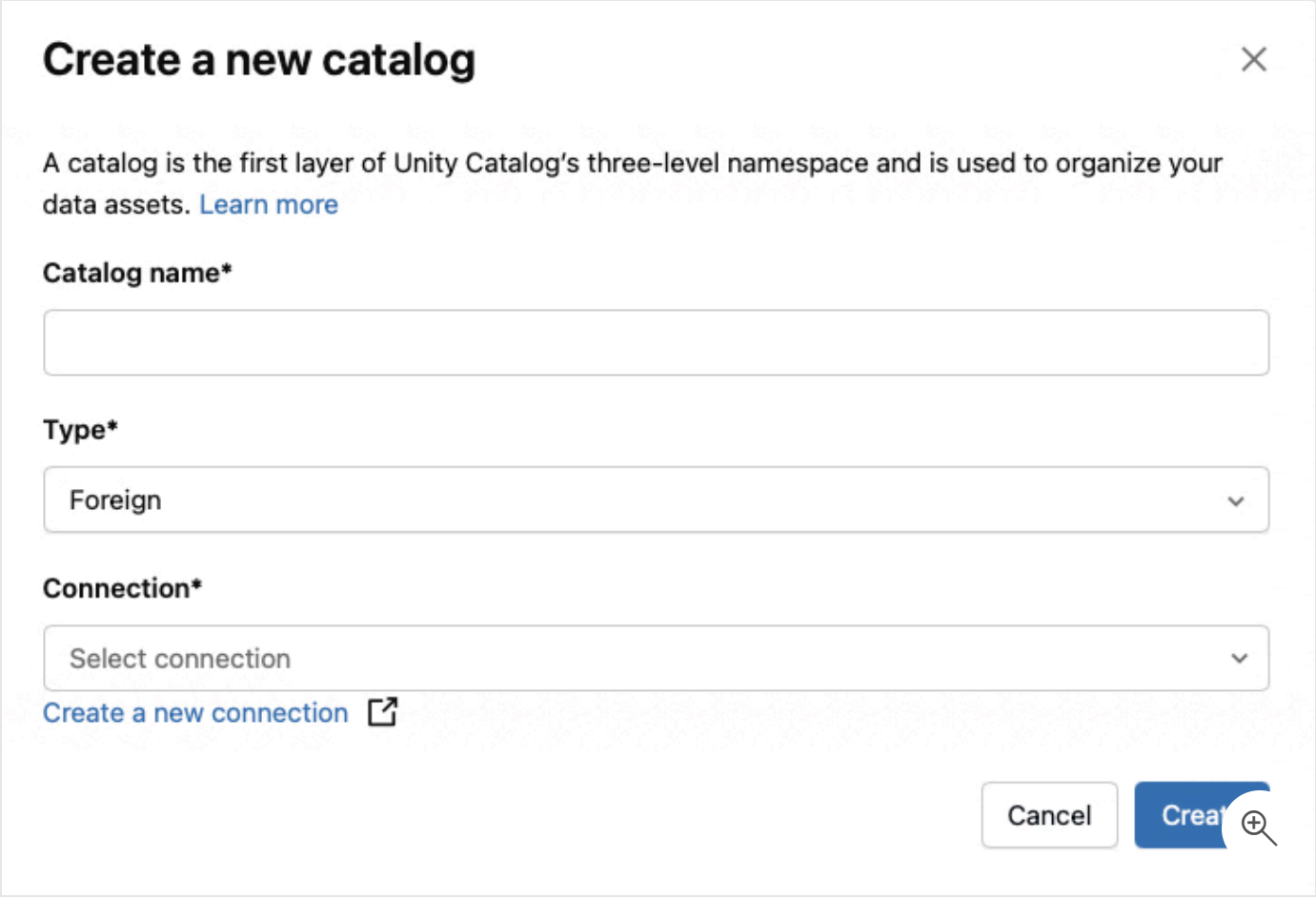
SQL

```
CREATE FOREIGN CATALOG IF NOT EXISTS prod_customer_data
USING CONNECTION postgresql_prod
OPTIONS (database 'customers');
```

This command creates a catalog named `prod_customer_data` that mirrors the `customers` database from your PostgreSQL server. The `IF NOT EXISTS` clause prevents errors when running the script multiple times. Unity Catalog immediately synchronizes metadata from the external database, making its schemas and tables visible in your workspace.

Using Catalog Explorer, you can create the foreign catalog during connection setup or afterward:

1. Select **Catalog** and then select **Create a catalog**.
2. Choose **Foreign catalog** as the catalog type.
3. Enter a name like `prod_customer_data`.
4. Select the connection you created earlier.
5. Specify the external database name (for example, `customers`).
6. Select **Create**.



Metadata synchronization happens automatically each time you interact with the foreign catalog. When you query `SELECT * FROM prod_customer_data.public.transactions`, Unity Catalog refreshes its view of the external schema structure before executing the query. This ensures you always see current metadata, even if database administrators add new tables or columns in the external system.

While automatic synchronization is optimal for most workloads, it can impact performance for high-frequency queries or when external systems are accessed by tools outside Databricks. For these scenarios, you can manually refresh metadata on a schedule using the `REFRESH FOREIGN` command. This approach allows queries to run faster using cached metadata rather than refreshing on every query. You can refresh at different levels of granularity:

SQL

```sql
-- Refresh an entire catalog
REFRESH FOREIGN CATALOG prod_customer_data;

-- Refresh a specific schema
REFRESH FOREIGN SCHEMA prod_customer_data.public;

-- Refresh a specific table
REFRESH FOREIGN TABLE prod_customer_data.public.transactions;
```

Schedule these refresh operations using Lakeflow Jobs to run at intervals that match how frequently your external schema changes. This proactive approach is useful immediately after creating a foreign catalog, when the first query would otherwise trigger a full metadata refresh.

Grant access and query foreign data

Foreign catalogs appear in Unity Catalog alongside your standard catalogs, making them subject to the same permission model. You control access by granting appropriate privileges to users and groups.

Grant read privileges to users who need to query foreign data:

SQL

```sql
GRANT USE CATALOG ON CATALOG prod_customer_data TO `data-analysts`;
GRANT USE SCHEMA ON SCHEMA prod_customer_data.public TO `data-analysts`;
```

```
GRANT SELECT ON TABLE prod_customer_data.public.transactions TO `data-analysts`;
```

These grants follow Unity Catalog's hierarchical permission model. Users need `USE CATALOG` on the catalog, `USE SCHEMA` on the schema, and `SELECT` on specific tables. This granular control lets you expose some foreign tables while restricting others, even though they all exist in the same external database.

Query foreign tables using standard SQL syntax. Once permissions are in place, users query foreign data exactly as they would query native Unity Catalog tables:

SQL

```
SELECT
  customer_id,
  SUM(transaction_amount) AS total_spent
FROM prod_customer_data.public.transactions
WHERE transaction_date >= '2024-01-01'
GROUP BY customer_id
ORDER BY total_spent DESC
LIMIT 100;
```

Azure Databricks translates this query into SQL appropriate for PostgreSQL, pushes the query down to the external database, and returns results. The external database performs the aggregation and filtering, leveraging its own compute resources while Azure Databricks handles the final result set.

Query execution uses remote compute in the external system. Understanding this behavior helps you optimize performance. Complex joins, aggregations, and filters are pushed down to the external database when possible, reducing data transfer. However, if your result set is very large, the Databricks executor retrieving data might run out of memory. Monitor query performance and consider using materialized views to cache frequently accessed external data locally.

Next unit: Configure AI/BI Genie instructions

Previous Next